

Analyse

This project is dedicated to the analysis of the Speed Dating Dataset using machine learning techniques. The primary goal is to predict whether two individuals will match based on their attributes and preferences.

We check what data we have, what distribution is there, and what features are important for our prediction task.

Conclusion: We decide to use only some of the features for our model, that are correlated with match target feature.

First try to model train

We try to train model using all the features that we have after analysing stage.

And we didnt get very good results. All results are provided in baseline report.

We now have baseline to improve our model.

From the results we can see that our model is overfitting a lot, our val_error is growing and our train_error is decreasing, aslo our test_error is increasing.

And to improve it we need to reduce number of features and increase number of samples that we can use for training and also try to tune hyperparameters of our model and try different configurations of model architecture.

Also we want to see one more statistic for our model with validation set.

And we found an issue that we fit_transform not only training set but also validation and test sets. So we fix it in the next try.

Also from the binary classification we can say we want sigmoid activation in the output layer and binary crossentropy loss function. And from our baseline we can see we want less features in first hidden layer.

1 Best runs

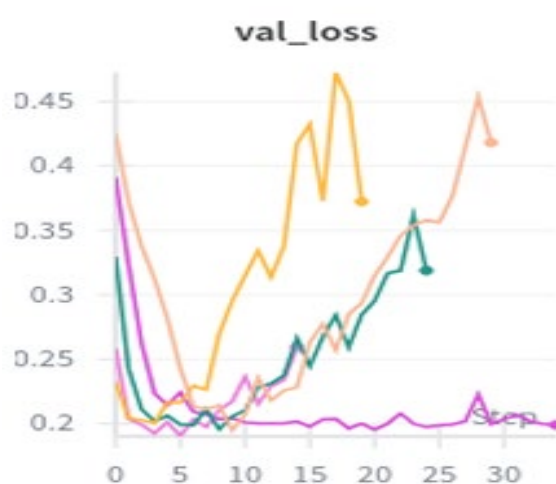


Figure 1:

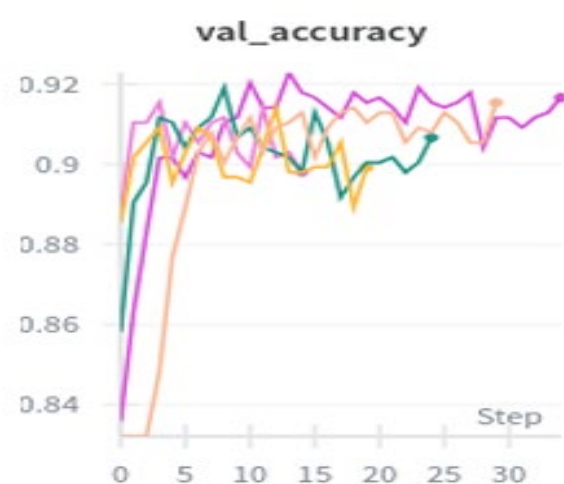


Figure 2:

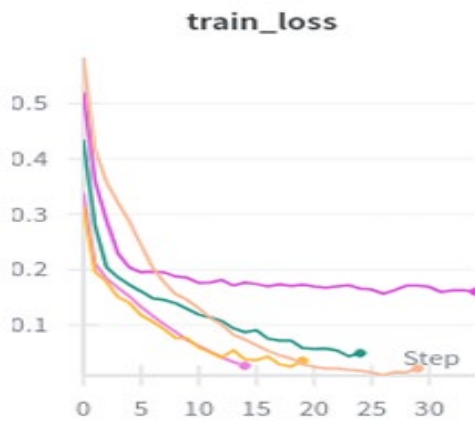


Figure 3:

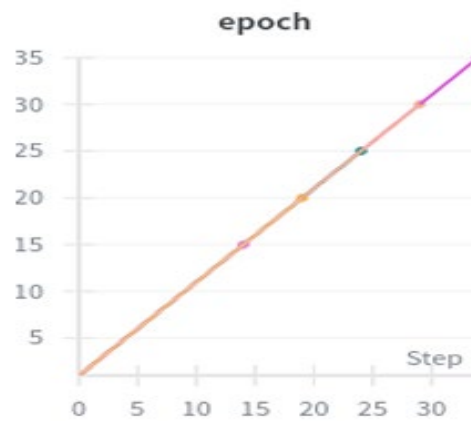


Figure 4:

Results: Our baseline shows that for that classifications we need better hyperparameters, because a lot of models are heavily overfitting. Also we can see that models are very unpredictable on value accuracy. Value accuracy is high, but final test shows accuracy around 0.83. That potentially mean of too high learning rate and very strong overfitting. We should make model smaller.

Second try to model train

We selected only some of the features that are correlated with match target feature and not with each other.

We also transform nan values using IterativeImputer from sklearn to increase number of samples that we can use for training.

We try to train model using these features:

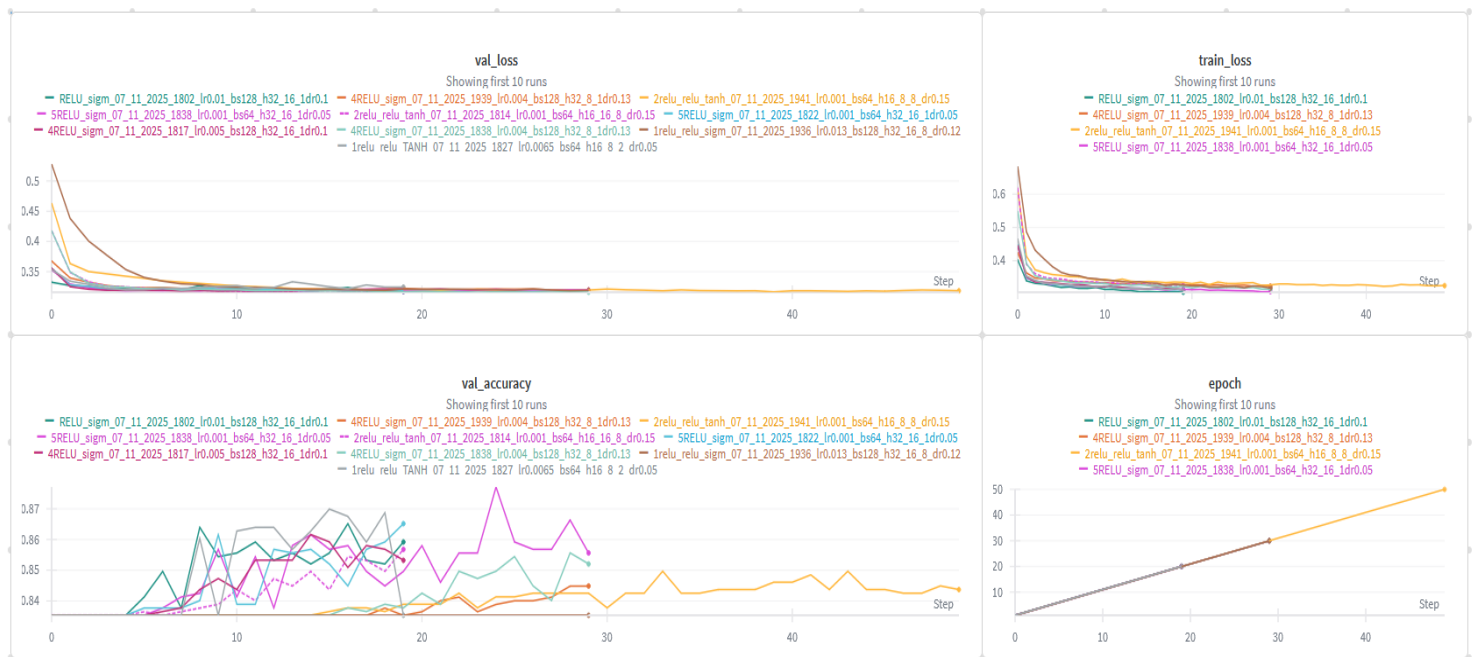
Also after getting initial results, we can see that model often peaks on 84 accuracy, and overfits. From that information and the fact that our data is heavily skewed on match = 0 (around 80 to 20), we can say that we need to alter our loss function. Also we decided to use sweep to increase number of hyperparameters tested out.

In our sweep model we define hypeparameter to be tested out: different hidden dims (from 4 to 128), batch sizes (From 32 to 512), learning rate (from 0.00001 to 0.01), dropout (from 0.0 to 0.5), activation functions (Gelu, Sigmoid, ReLu, Tanh) and amount of hidden layers (from 2 to 4).

For sweep, we will use random method and maximize val_accuracy.

Results: Val_accuracy unpredictable and unstable. Among top 10, there are still models that are having accuracy of 0.83 (Exact proportions of 0 to 1 math.) That means that are among top 10 results are still models that are giving only zero, and val_loss and train_loss are changes of the "confidence" but not the result. Other models dont have exact way to predict 1, so they ocasionaly predict it, by random. Thats why there is no gradually accending function.

Best result of final_test_accuracy - 0.848, final_test_loss - 0.315



Third try to model train

We changed loss function by multiplying the target 1 on the proportions of match = 0 to match = 1. In that way model will not predict only zeroes, but will heavily try to predict ones but with less accurate results, by punishing the incorrect Match = 1 much more heavily than incorrect Match = 0.

Results of sweep and manual testing showed us that model with three and four layers are overfitting heavily, and sweep tries to model them with dropout in range of 0.3 to 0.5. That can be unpredictable and not efficient.

In the next sweeps we will use less 3 layered models and no 4 layers models.

Also we diagnosed that most of the hidden layers should be RELU or GELU and last layers can be either Sigmoid, or Tanh, or Also ReLU.

Sigmoid and Tanh functions are underperforming when used multiple times.

Models with 2 layers with range of neurons on first layer around 32 - 16 shows good results with less dropout than 0.22. After sweep we fine tune the best models manually changing some of the parameters and watching how test_loss and val_loss performs.

If Val_loss raises after some training, we increase drop out, or put less neurons. If it is possible that model underperforms and can still more, we give little more epochs, and try out bigger learning rate.

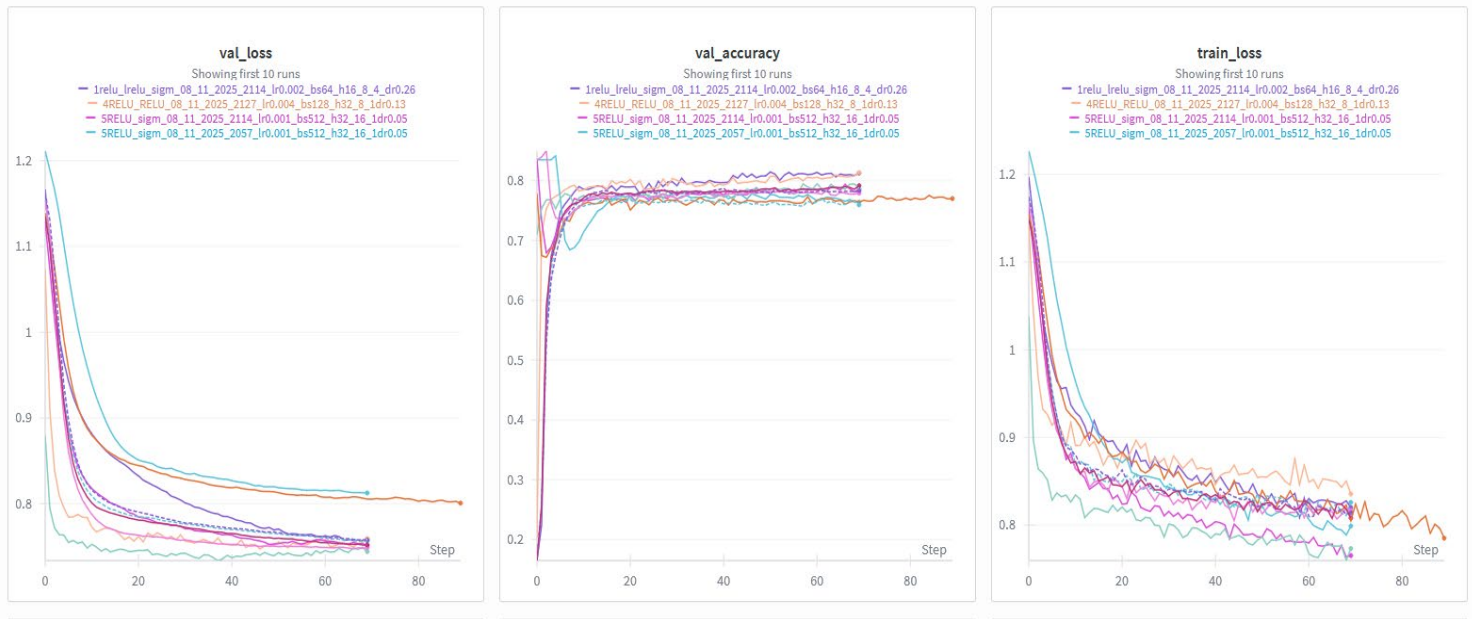
After changing the loss function, model lost its accuracy from 0.83 to 0.77-0.79. As we figured out, previous models tended to produce only zeroes, and very rarely or never at all gave answer 1. That's why model was overall more accurate but useless. New model training on producing accurate 77% of answers right both on 0 and 1.

Results

Val_accuracy became much more stable. As we can see, models tend to perform better at start, until drop. We deduce that in initial epochs, data may contain very little or no match = 1. After first few epoch we expect raising number of match = 1 and the drop of accuracy.

We tested different amount of epochs, but it tended only to overfit the models. Best one perform with epochs around 60-70.

Names of the runs may not describe the parameters of the Models.



Final test accuracy – 0.798. Model –ReLu Relu Sigmoid. Hidden dims: 32, 16,4. Learning rate – 0.0016. Batch size 256. Dropout 0.21.

Final test accuracy – 0.782. Model – Relu Relu. Hidden dims: 32 8. Learning rate – 0.002. Batch size 128.Drop out 0.25.

Final test accuracy – 0.772 Model – Relu Relu Sigmoid. Hidden dims: 32, 16, 8. Learning rate – 0.0025. Batch size 512. Dropout 0.15